



ARM Toolchain

F. Angione, P. Bernardi, R. Ferrero,

Politecnico di Torino

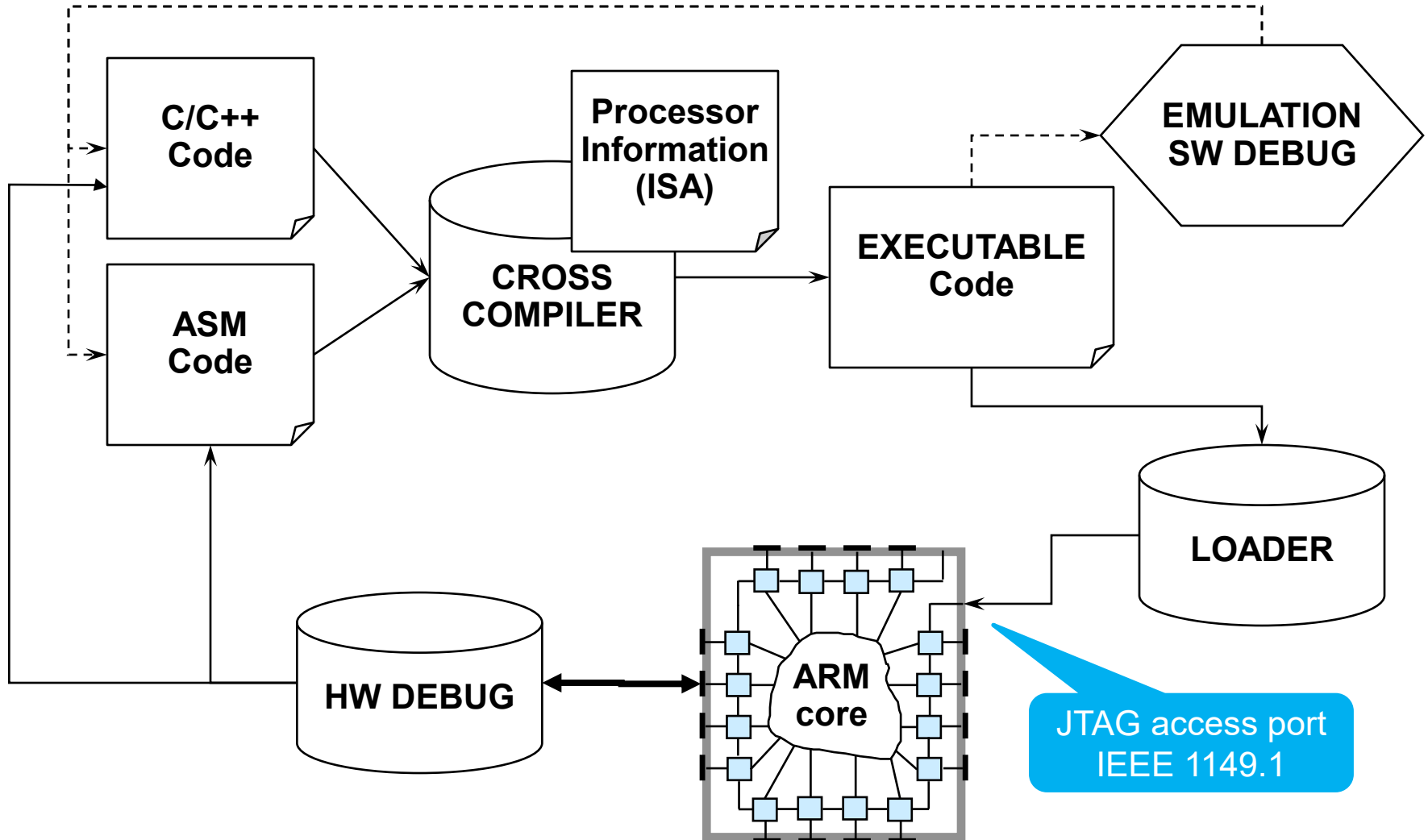
Dipartimento di Automatica e Informatica (DAUIN)

Torino - Italy

This work is licensed under the Creative Commons (CC BY-SA) License. To view a copy of this license, visit <http://creativecommons.org/licenses/by-sa/3.0/>

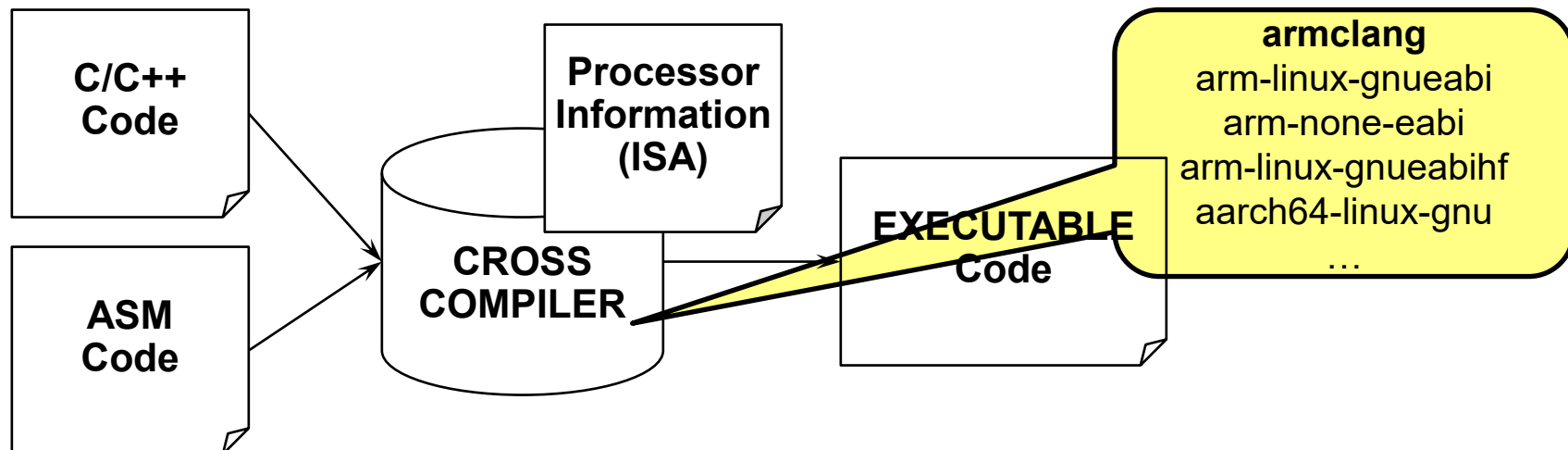


Toolchain



The cross-compiler

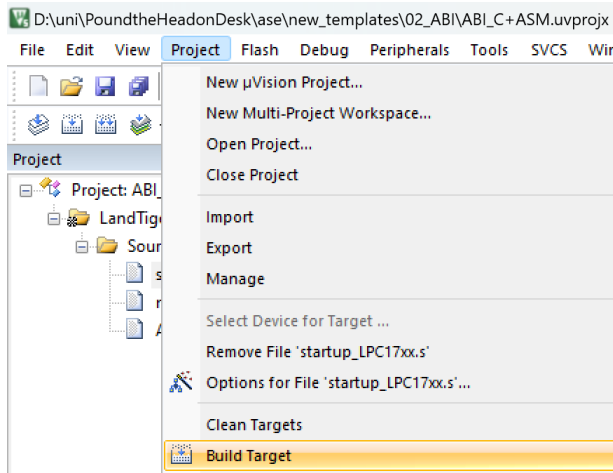
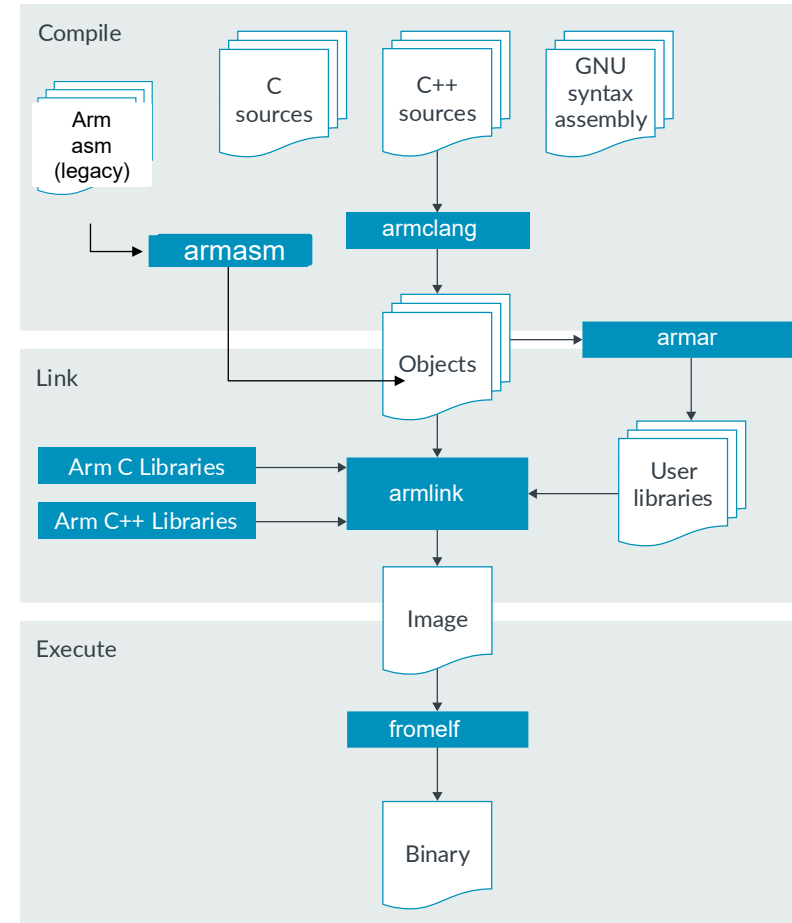
- It creates executable code for a platform other than the one on which it is running.
 - Includes a set of programming tools (toolchain)
 - Preprocesses the source code.
 - Translates high level code in machine code.
 - Introduces already developed library.



Arm Toolchain

- Composed of 3 phases:
 - 1) preprocessing + compilation
 - 2) link
 - 3) execute
- The toolchain components are highlighted in blue

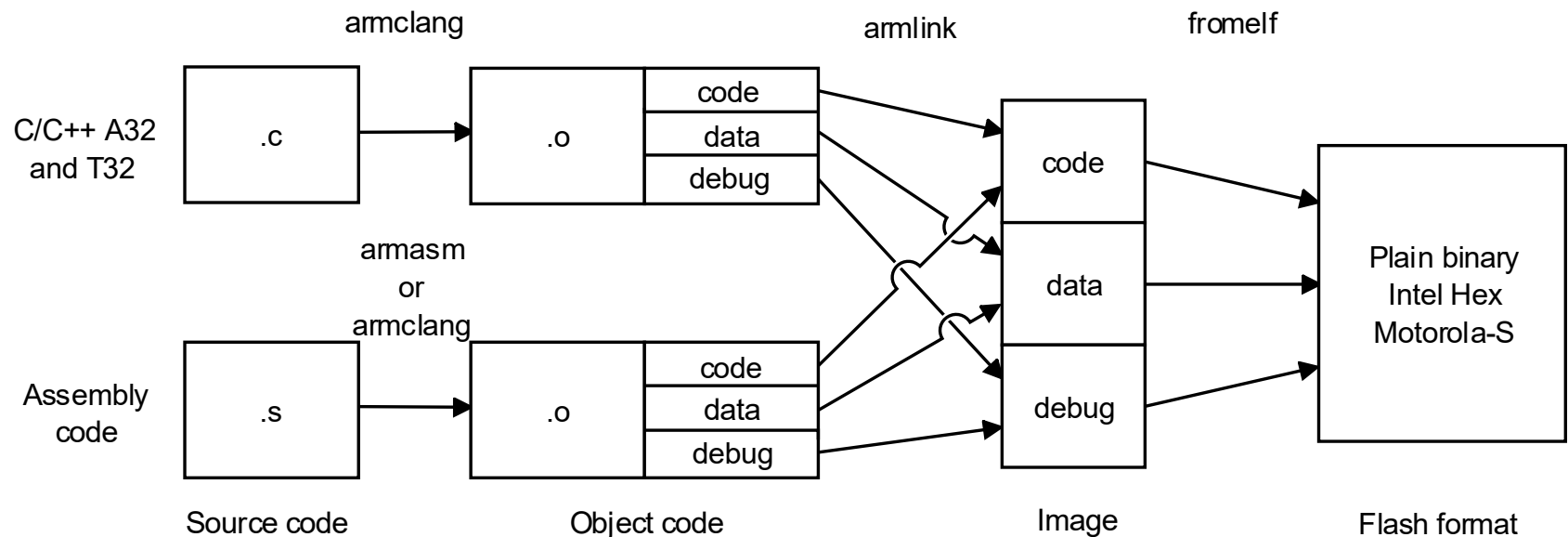
<https://developer.arm.com/Tools%20and%20Software/Arm%20Compiler%20for%20Embedded>



- All the three phases are included when building the project.

1) Compile phase

- The compile phase create object files (.o):
 - *armasm* for legacy support
 - *armclang*, based on an enhanced version of *clang* (frontend) and *llvm project*, with proprietary customizations <https://clang.llvm.org/>

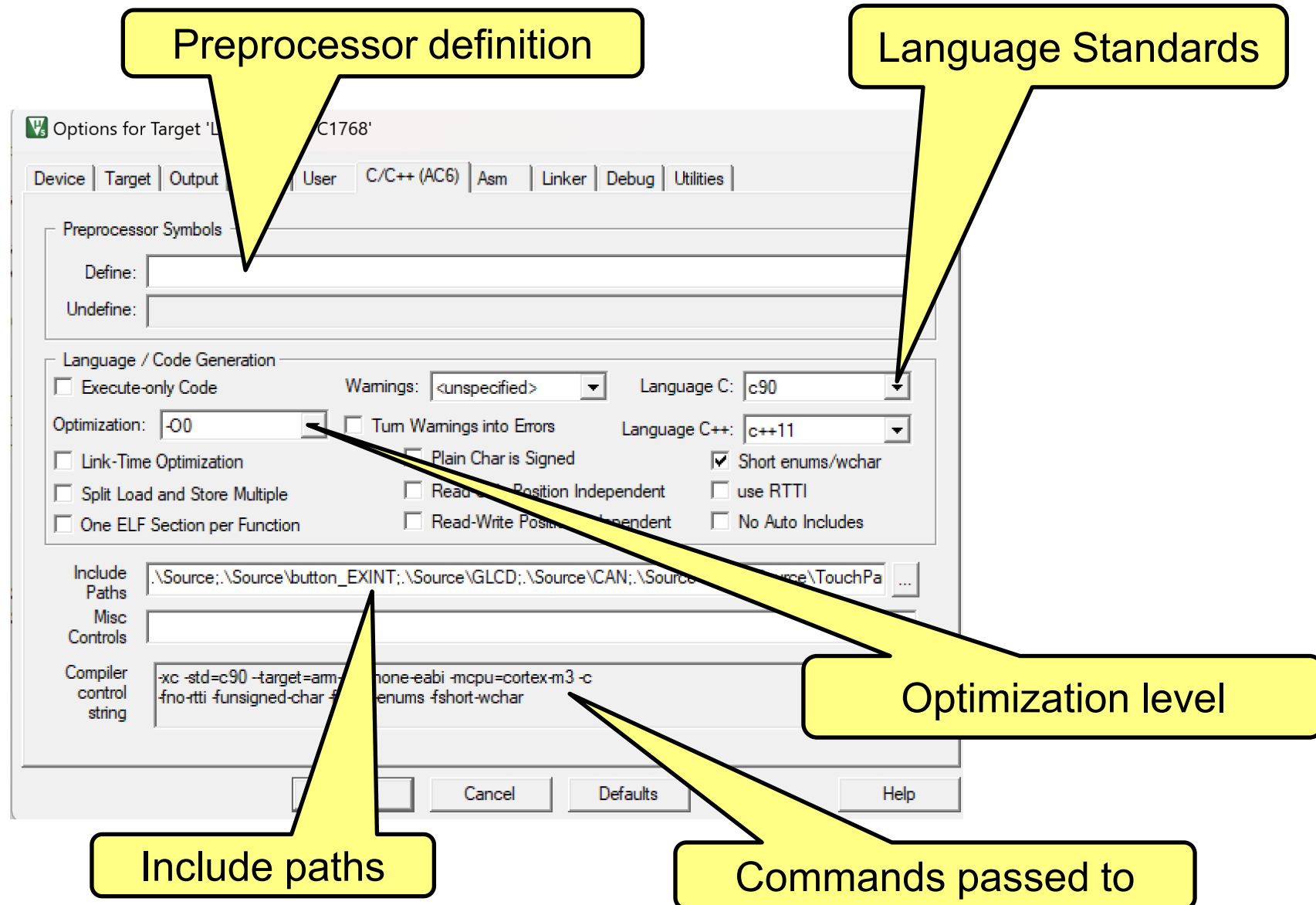


armclang Vs armasm

- *armclang* is used to:
 - compile high level code such as c or c++
 - assemble assembly language code, or inline assembly, written in GNU syntax.
- armasm is used to:
 - assemble assembly code in armasm syntax
- Differences of assembly syntax:

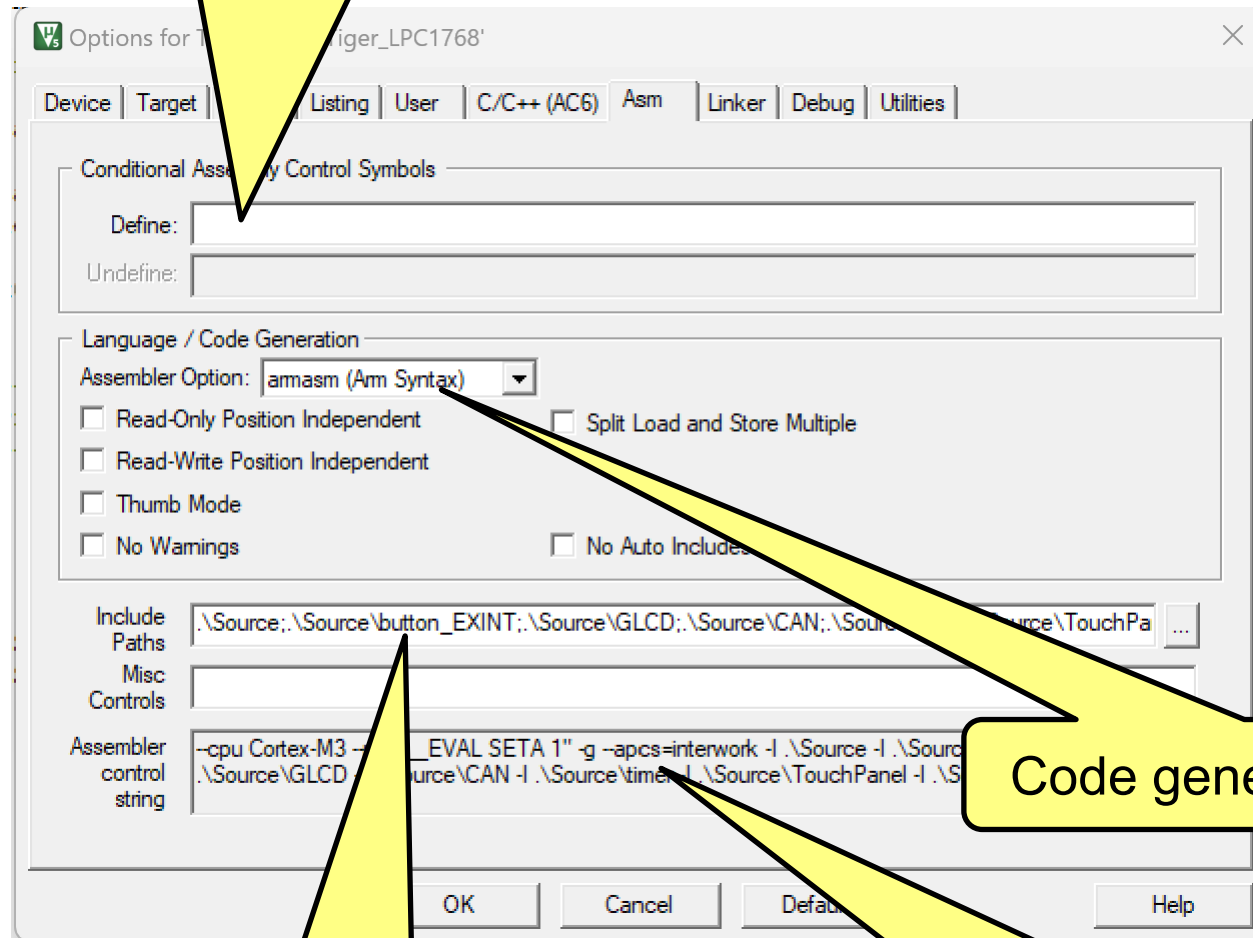
<https://developer.arm.com/documentation/100068/0623/Migrating-from-armasm-to-the-armclang-Integrated-Assembler/Overview-of-differences-between-armasm-and-GNU-syntax-assembly-code>

Compile phase: armclang6



Compile phase: armasm

Preprocessor definition



Code generation options

Include paths

Commands passed to

C Language Standard Selection

- Example: on the fly variable declaration

```
1
2
3
4 volatile int my_array[N];
5 int main(void) {
6
7     int i;
8
9     for (i=0;i<N;i++){
10        my_array[i]=i*3;
11    }
12    while(1);
13 }
```

```
1
2
3
4 volatile int my_array[N];
5 int main(void) {
6
7     for ( int i=0;i<N;i++){
8         my_array[i]=i*3;
9     }
10
11    while(1);
12 }
13
```

- The on the fly variable declaration gives a warning: see the build log

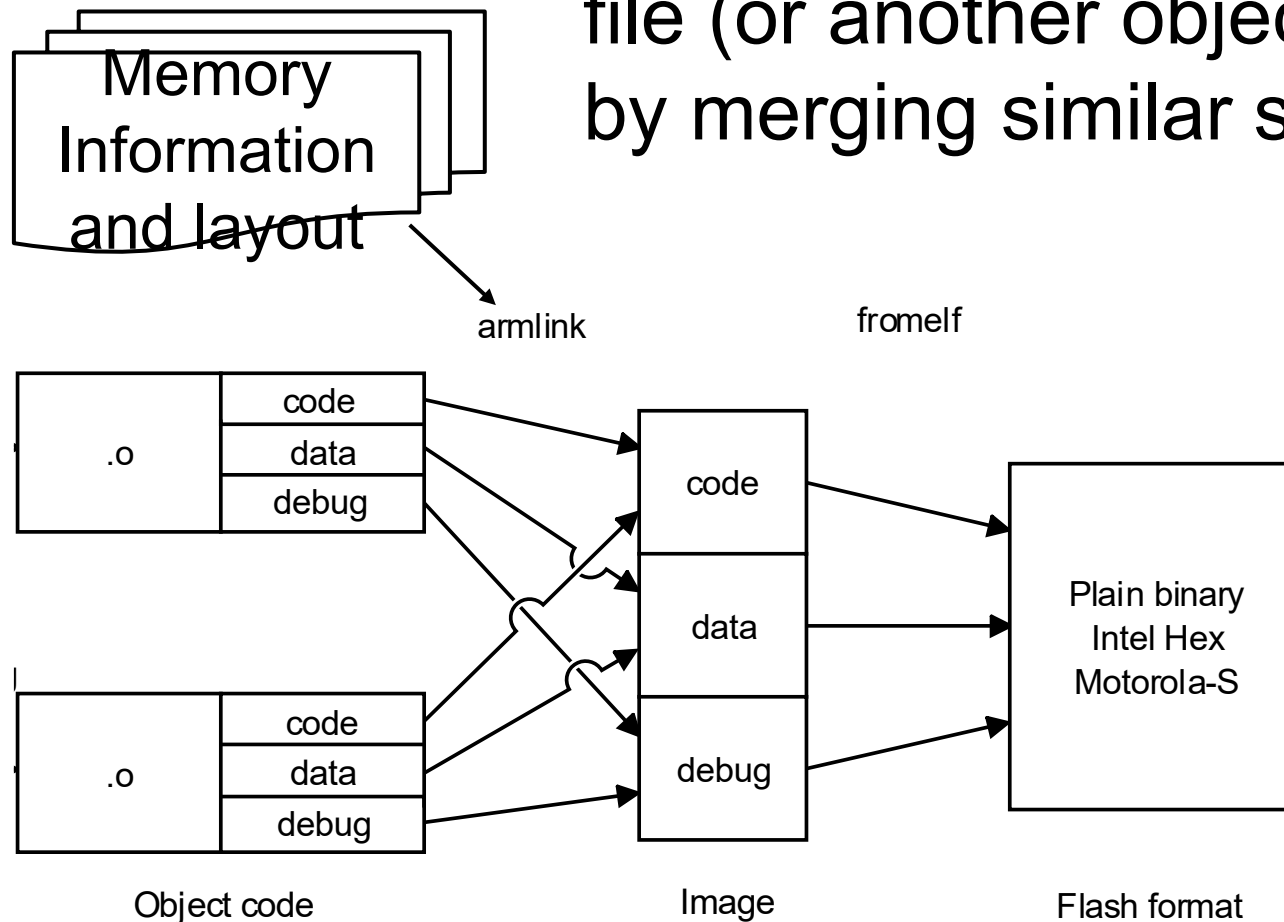
```
*** Using Compiler 'V6.22', folder: 'D:\programmi\keil\ARM\ARMCLANG\Bin'
Rebuild target 'LandTiger_LPC1768'
assembling ASM_func.s...
assembling startup_LPC17xx.s...
creating list file for main.c...
creating preprocessor file for main.c...
Source/main.c(8): warning: GCC does not allow variable declarations in for loop initializers before C99 [-Wgcc-compat]
8 |         for ( int i=0;iSoftware Packages used:
```

Optimization levels

- They strongly affect the performance and size of executables (and **machine code** in executable).
 - Level 0 (O0): Turns off most optimizations. Generated code that directly corresponds to the source code.
 - Level 1 (O1): Restricted optimization. The best debug view for the trade-off between image size, performance, and debug.
 - Level 2 (O2): High optimization. The debug view might be less satisfactory.
 - Level 3 (O3): Very high optimization, poor debug view.
 - Fast (Of), Max (Omax), Size (Os - Oz).

2) Link phase: armlink

- Link all object files into a single executable file (or another object file) by merging similar sections.

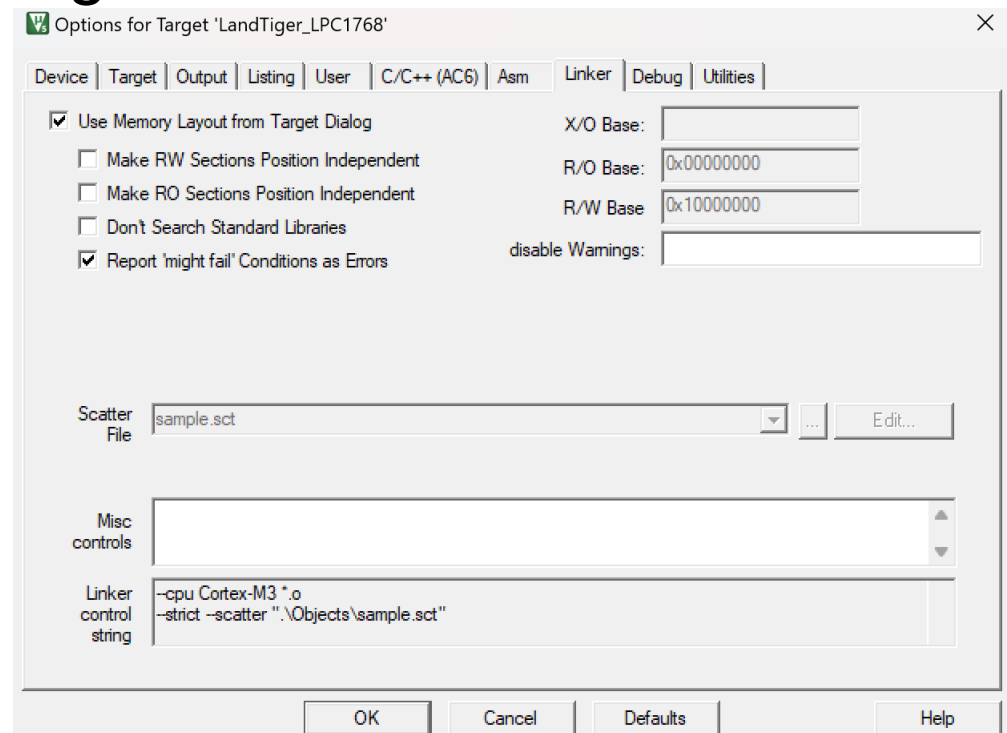


Link phase: armlink

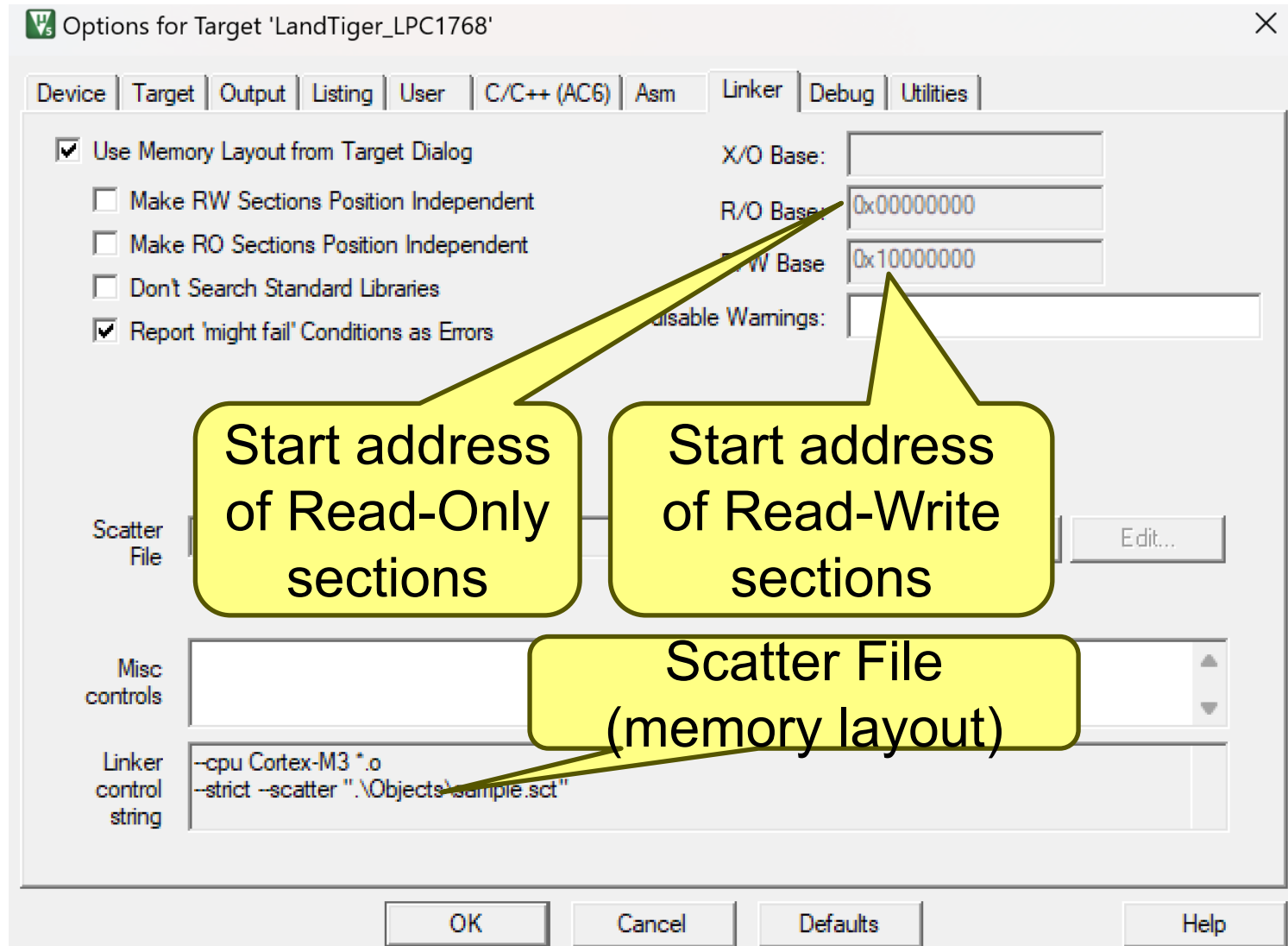
- It resolves:
 - Functions and variables (their symbols/labels are substituted with addresses).
 - Linker symbol (different from functions and variables).
- It eliminates unused sections regardless of the optimization level:
 - It removes unreachable code and data from the final image.
- It needs memory information to organize the image memory layout.

Memory Information and layout

- You can specify the entry point at the startup (i.e., the function called at the system boot).
- You can specify the memory information:
 - By command line using armlink tool.
 - With a scatter file.
- You can specify additional custom code and data sections.



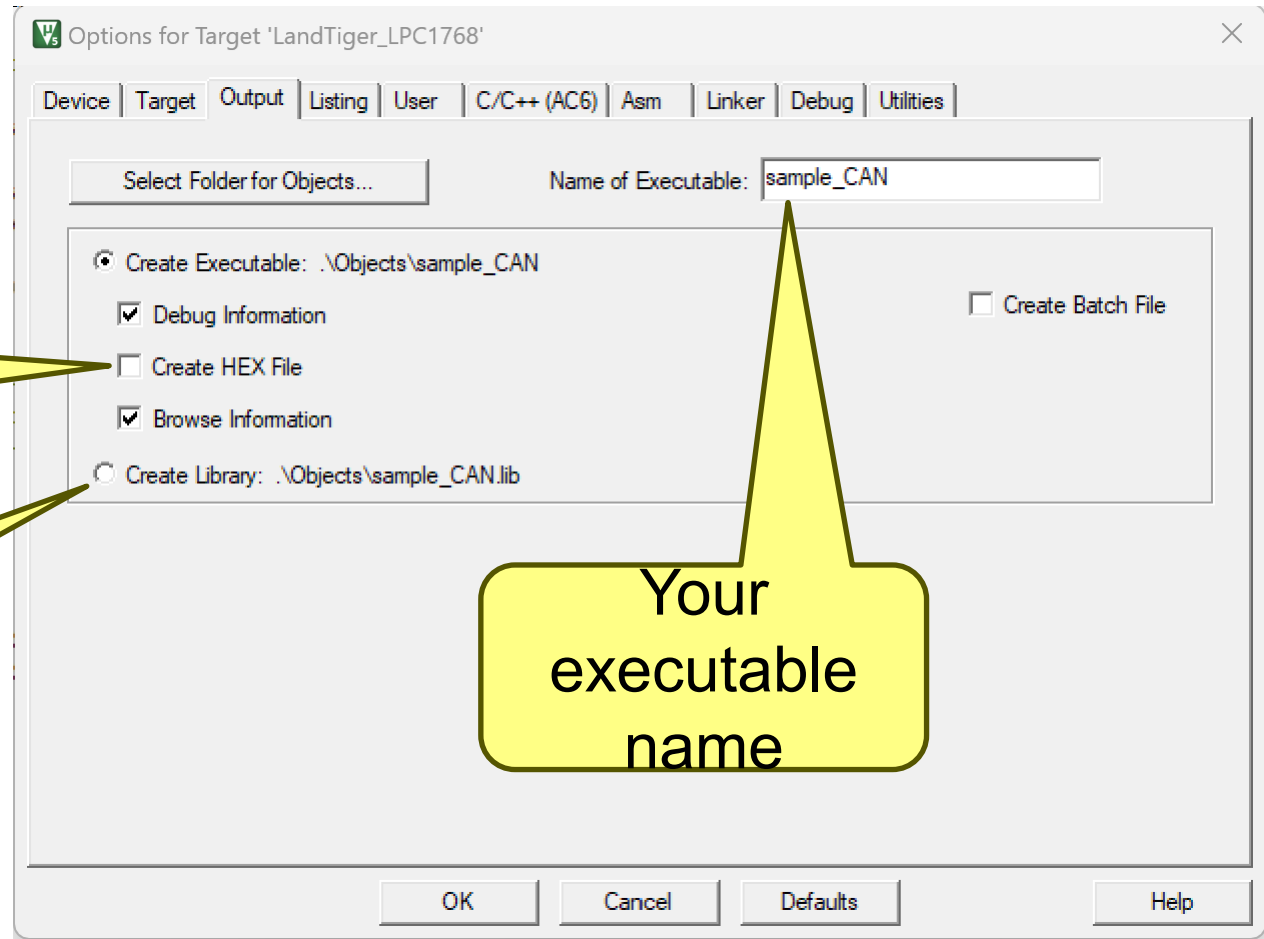
Memory Information and layout



Execute phase: fromelf

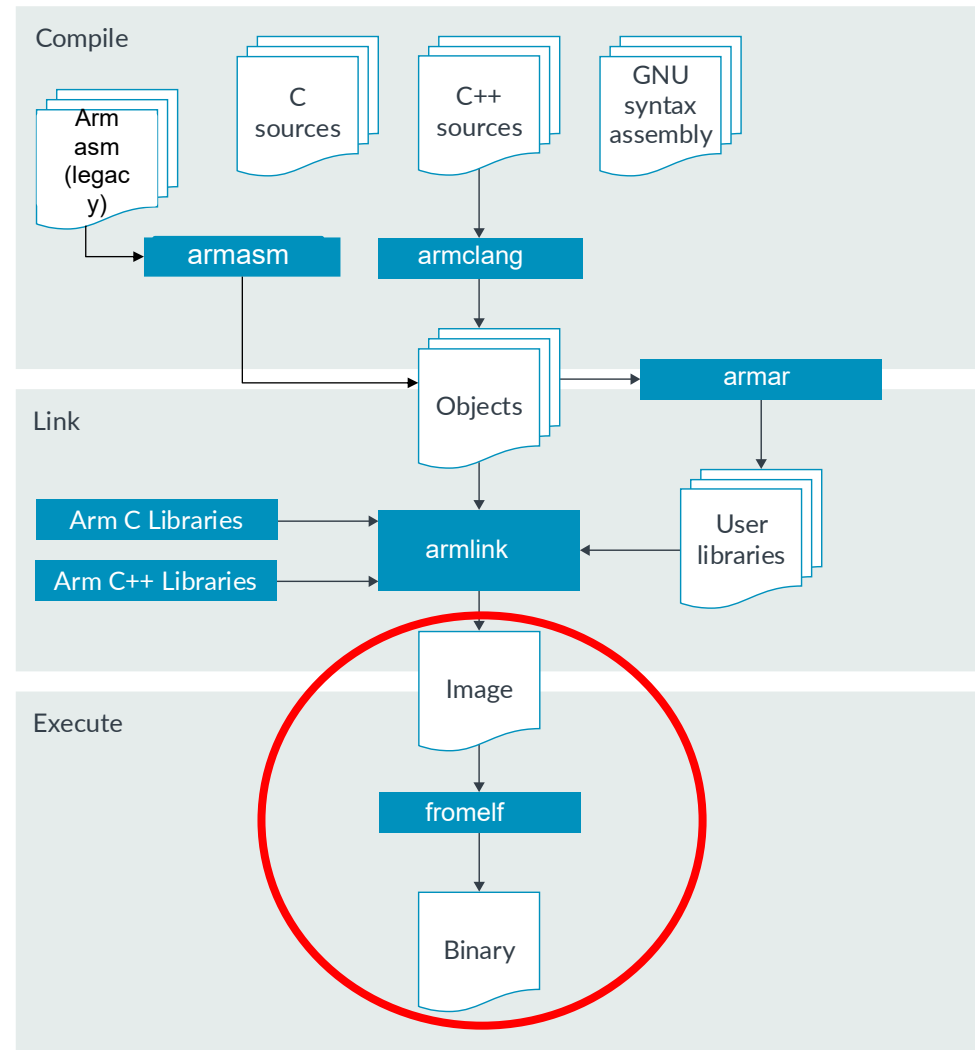
If you want a
hex file

If you want a
library to be
included into
another project



Compilation output files

- The Arm toolchain produces:
 - 1) the executable file
 - 2) the listing and dependencies files
 - 3) the map file
 - 4) the static call graph file
 - 5) the build log file



1) Executable file

- The overall image (from the source code) is converted into an executable (.exe, .elf, .axf for Arm).
- Data and Code sections are in the executable.
- The executable is stored in ROM

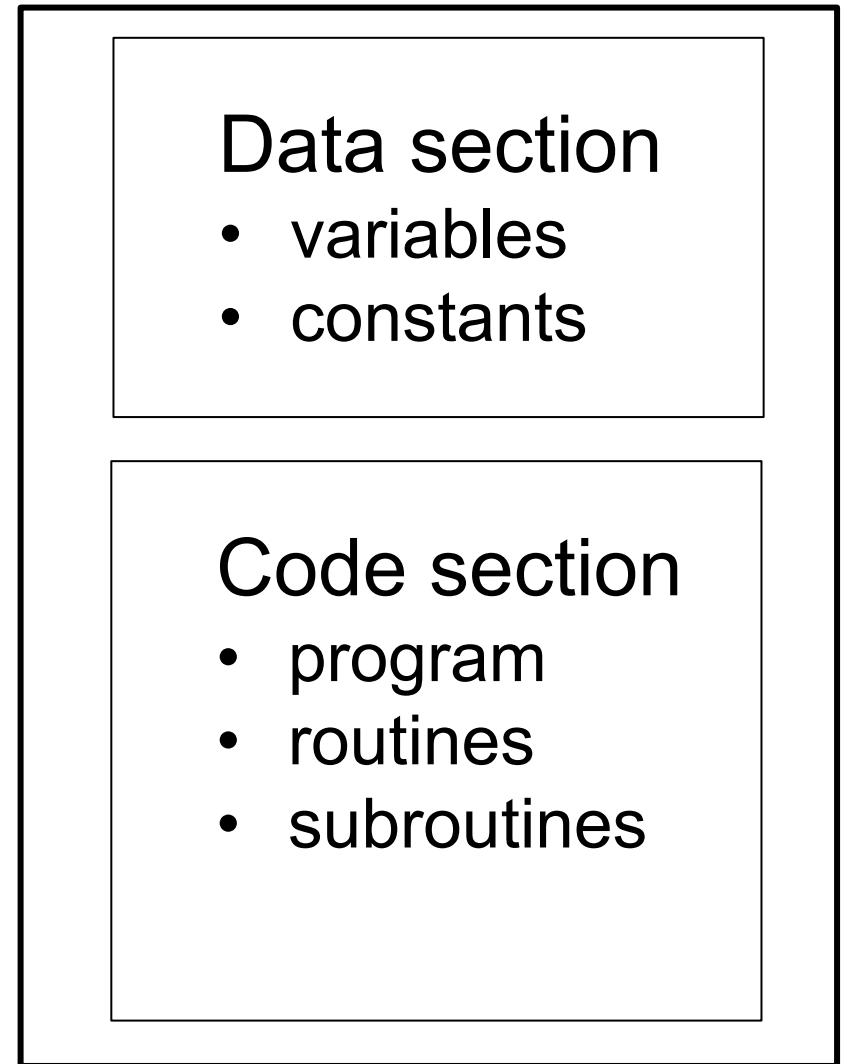
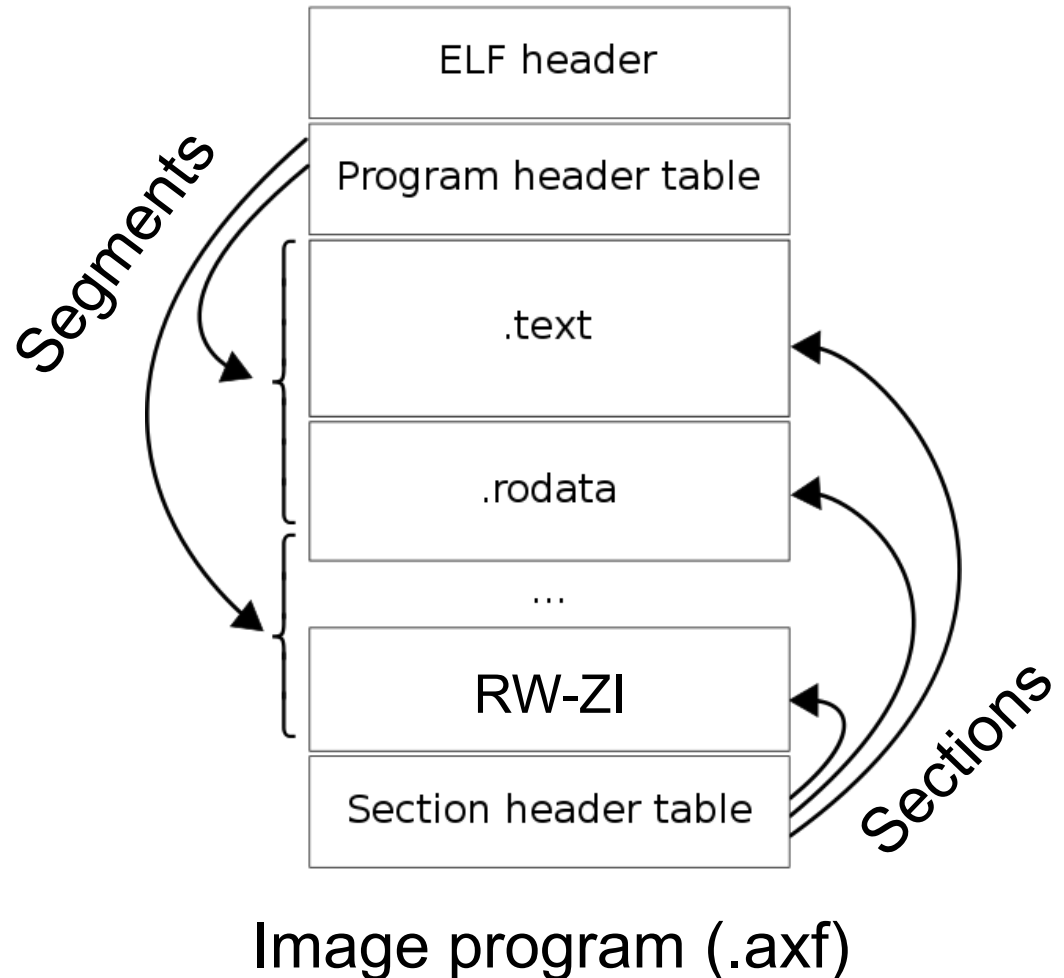


Image program (.axf)

Load view of the executable file

- Composed of:
 - entry address
 - stack and heap information
 - sections, used by the linker
 - segments, used by the loader (at runtime).



Execution view

- Before the image is executed:
 - executable segments are moved from ROM to their execution addresses in RAM.
 - RW data are copied from its load address in the ROM to its execution address in the RAM.
- Runtime memory layout information is calculated offline:
 - execution address and size of stack and heap

Execution view

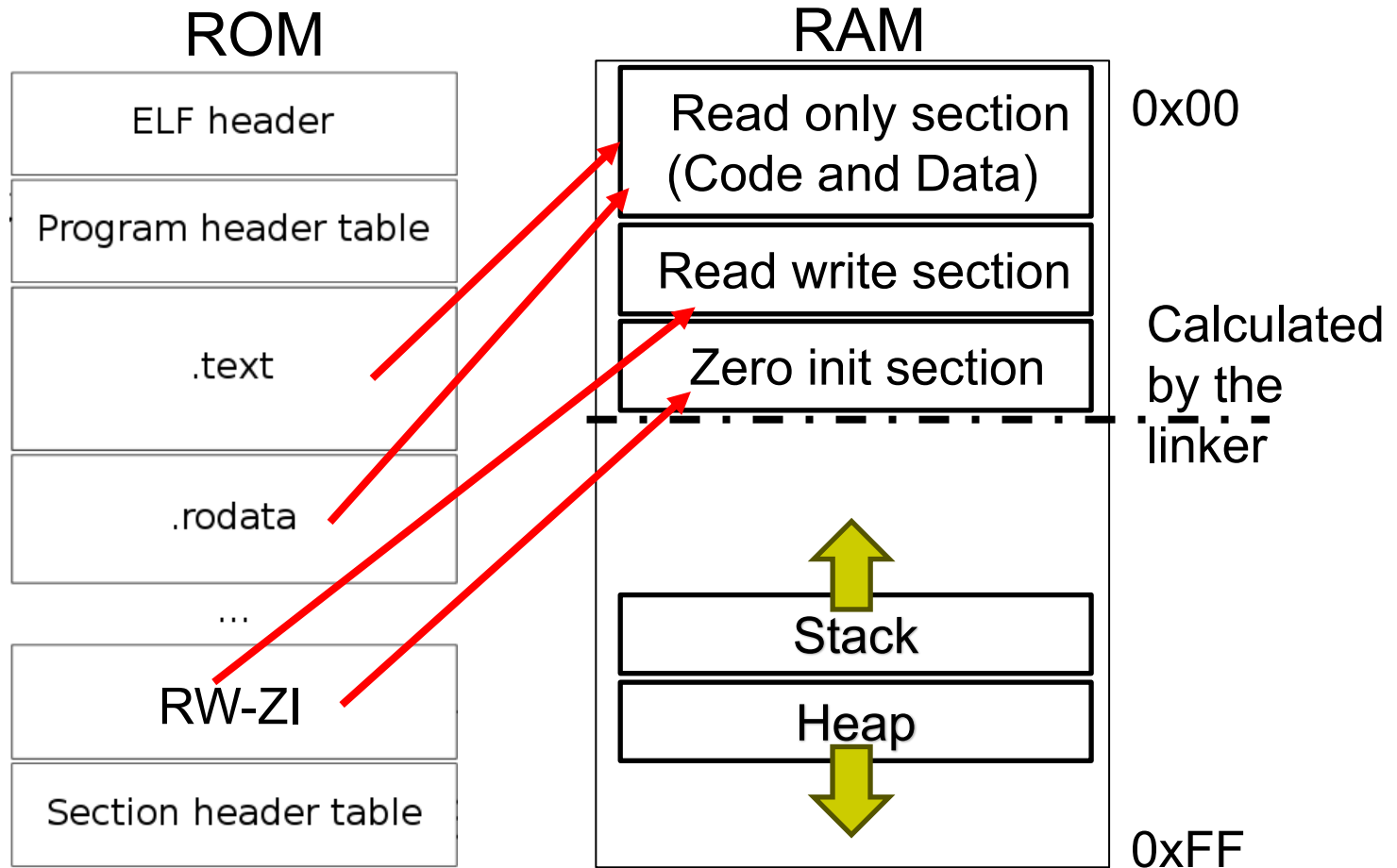
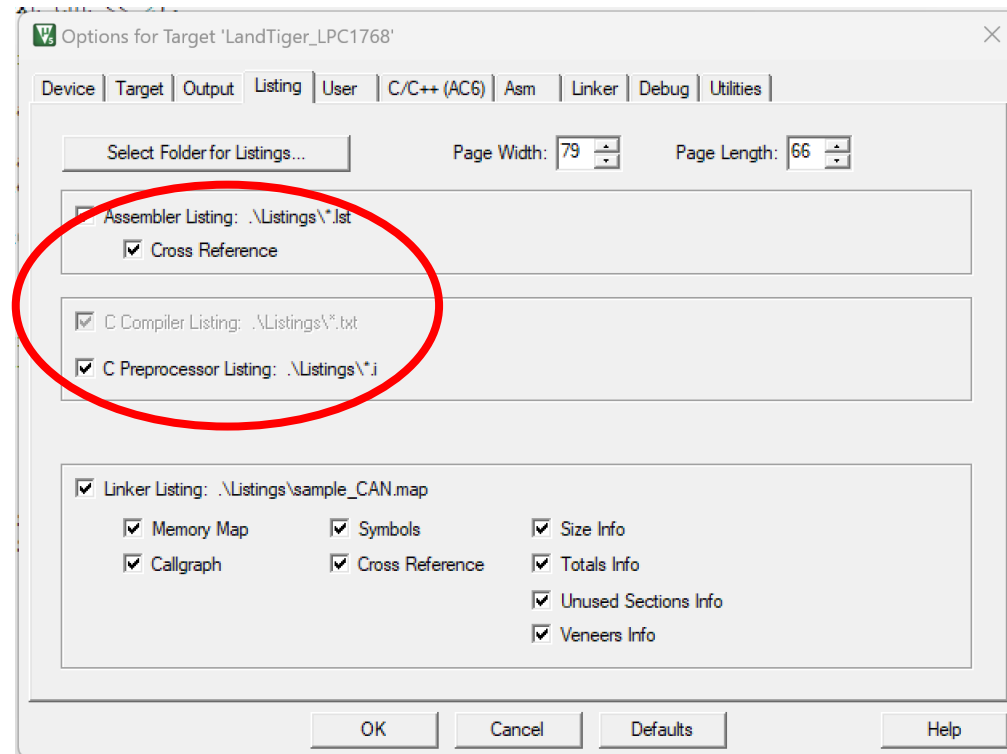


Image program (.axf)

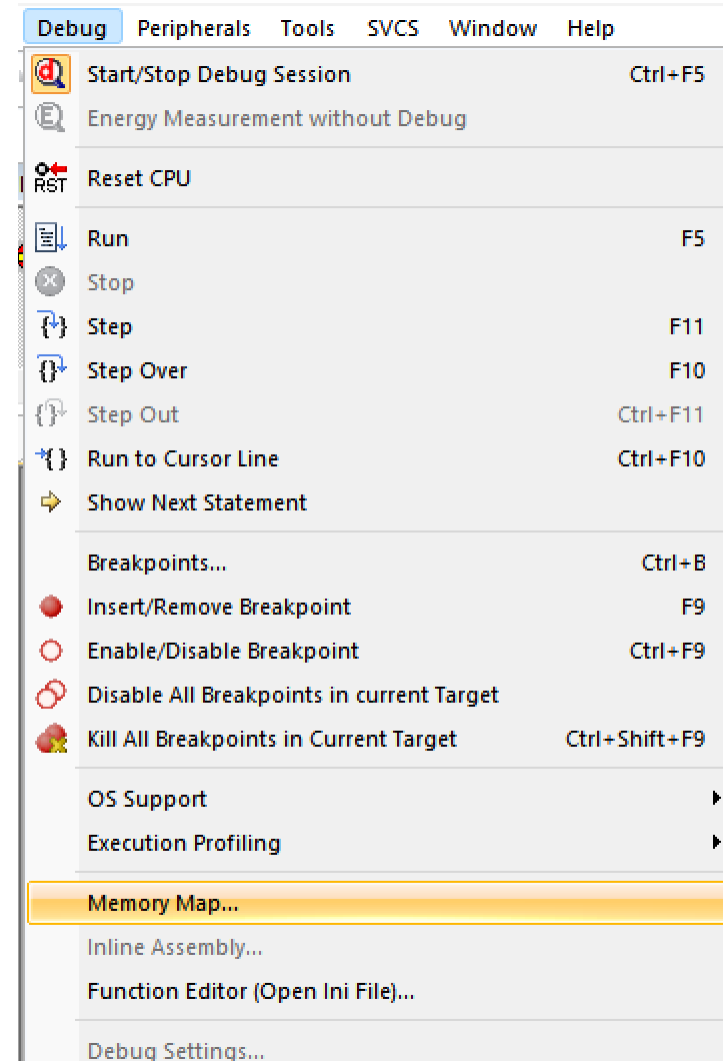
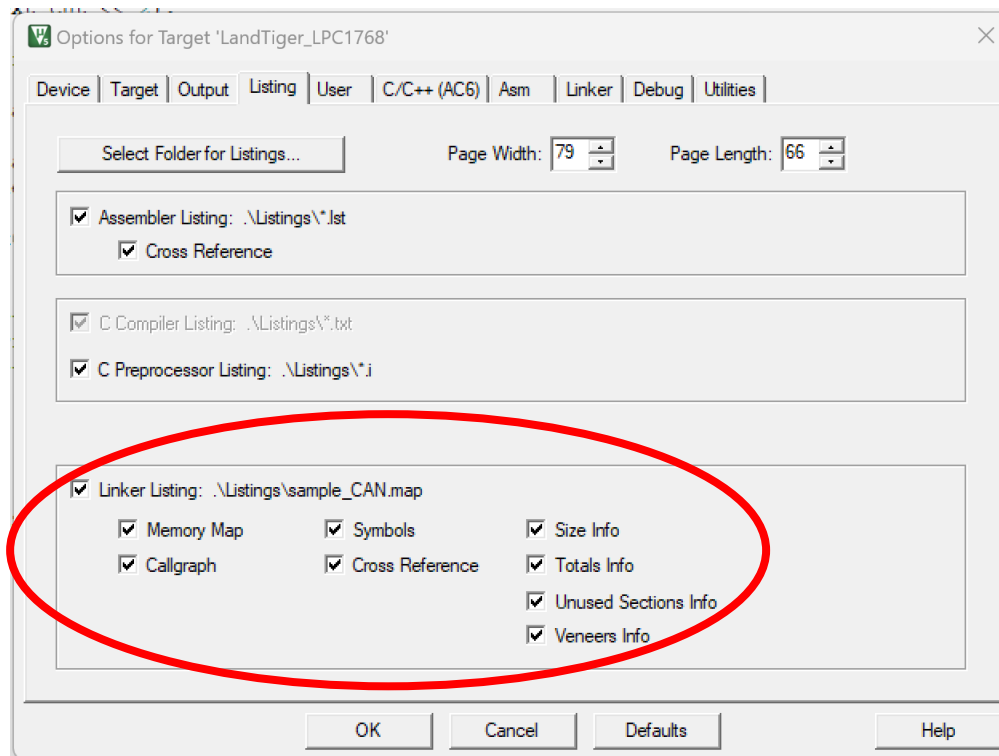
2) Listing and dependencies files

- Listing files are debugging files showing how the code is translated in machine code.
- The project dependencies files are generated (.d) and used by the toolchain
 - some information is needed during the link phase
 - the startup object depends only on the startup assembly file
 - the dependencies are in the .dep file.



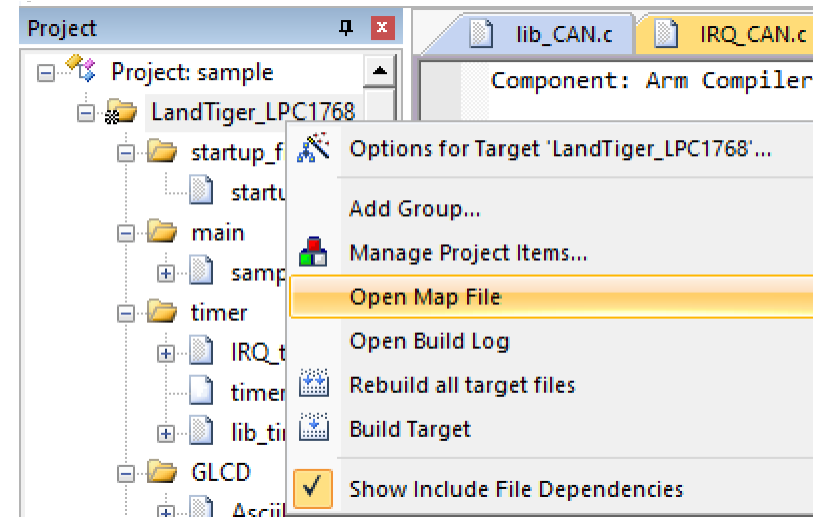
3) The map file

- It is the output “log” of the link phase
- It may be used by debugging tools



Content of the map file

- The map file includes:
 - cross references
 - image symbol table
 - memory map of the image
 - image component sizes
- The image symbol table reveals where the compiler is putting the variables

A screenshot of a text editor window showing the 'exceptions.map' file. The title bar indicates the file is 'exceptions.map'. The content is titled 'Image Symbol Table' and lists 'Local Symbols'. Below this is a table with columns: 'Symbol Name', 'Value', 'Ov Type', 'Size', and 'Object(Section)'.

Symbol Name	Value	Ov Type	Size	Object(Section)
.text	0x000000cc	Section	472	startup_lpc17xx.o(.text)
myVar1	0x000001d0	Data	1	startup_lpc17xx.o(.text)
myVar2	0x000001d1	Data	1	startup_lpc17xx.o(.text)
.text	0x000002a4	Section	0	heapauxi.o(.text)
.ARM.__at_0x02FC	0x000002fc	Section	4	startup_lpc17xx.o(.ARM.__at_0x02f

4) The static call graph file

- It represents the calling relationships between functions in the executable.
 - another debugging output “log” of the link phase
- It is a control-flow graph
 - saved in Objects folder
 - it has the same name of your project
 - it has extension .htm

Static Call Graph for image .\ABI_C+ASM.axf

#<CALLGRAPH># ARM Linker, 5060960: Last Updated: Sun Dec 03 13:41:14 2023

Maximum Stack Usage = 16 bytes + Unknown(Functions without stacksize, Cycles, Untraceable Function Pointers)

Call chain for Maximum Stack Depth:

__rt_entry_main ⇒ main

Functions with no stack information

- [_user_initial_stackheap](#)
- [ASM_func](#)

Mutually Recursive functions

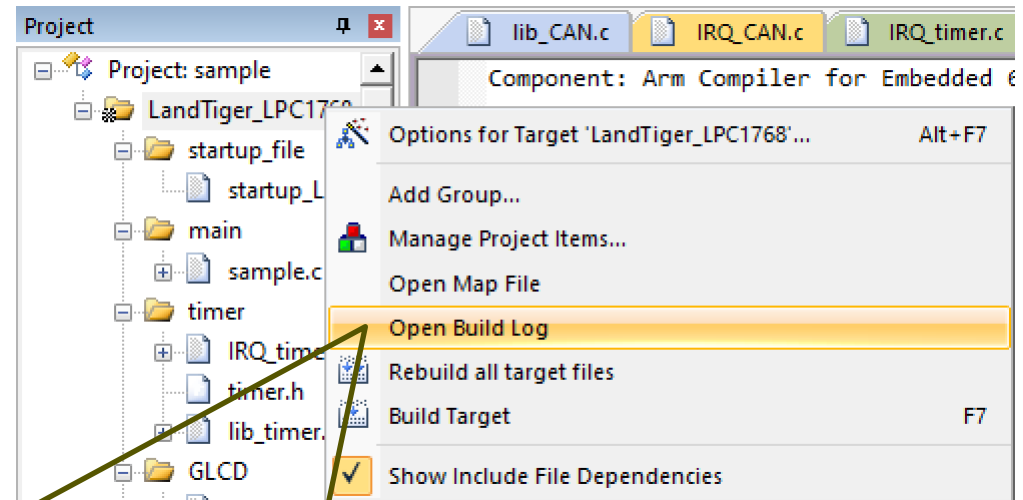
- [NMI_Handler](#) ⇒ [NMI_Handler](#)
- [HardFault_Handler](#) ⇒ [HardFault_Handler](#)
- [MemManage_Handler](#) ⇒ [MemManage_Handler](#)
- [BusFault_Handler](#) ⇒ [BusFault_Handler](#)
- [UsageFault_Handler](#) ⇒ [UsageFault_Handler](#)
- [SVC_Handler](#) ⇒ [SVC_Handler](#)
- [DebugMon_Handler](#) ⇒ [DebugMon_Handler](#)
- [PendSV_Handler](#) ⇒ [PendSV_Handler](#)
- [SysTick_Handler](#) ⇒ [SysTick_Handler](#)
- [ADC_IROHandler](#) ⇒ [ADC_IROHandler](#)

Function Pointers

- [ADC_IROHandler](#) from startup_lpc17xx.o(text) referenced from startup_lpc17xx.o(RESET)
- [ASM_func](#) from asm_func.o(asm_functions) referenced from asm_func.o(asm_functions)
- [BOD_IROHandler](#) from startup_lpc17xx.o(text) referenced from startup_lpc17xx.o(RESET)

5) The build log file

- The log of the entire build process includes:
 - tools versions
 - project name and date
 - output outcome
 - software packages
 - component files.



```
*** Using Compiler 'V5.06 update 7 (build 960)', folder: 'D:\programmi\keil\ARM\ARMCLANG5\Bin'
Rebuild target 'Target 1'
assembling ASM_func.s...
compiling main.c...
assembling startup_LPC17xx.s...
linking...
Program Size: Code=384 RO-data=384 RW-data=0 ZI-data=608
".\ABI_C+ASM.axf" - 0 Error(s), 0 Warning(s).
```

Program Startup on a System-on-Chip



Power On - Reset

startup.s

CPU executes
Reset_Handler
(fixed address)

runtime (rt) lib Arm

Prepare code and data
for the Main

```
148 Reset_Handler  PROC
149                EXPORT Reset_Handler
```

```
LDR    R0, =__main
BX     R0
ENDP
```

Main()

ROM

Executable
Code

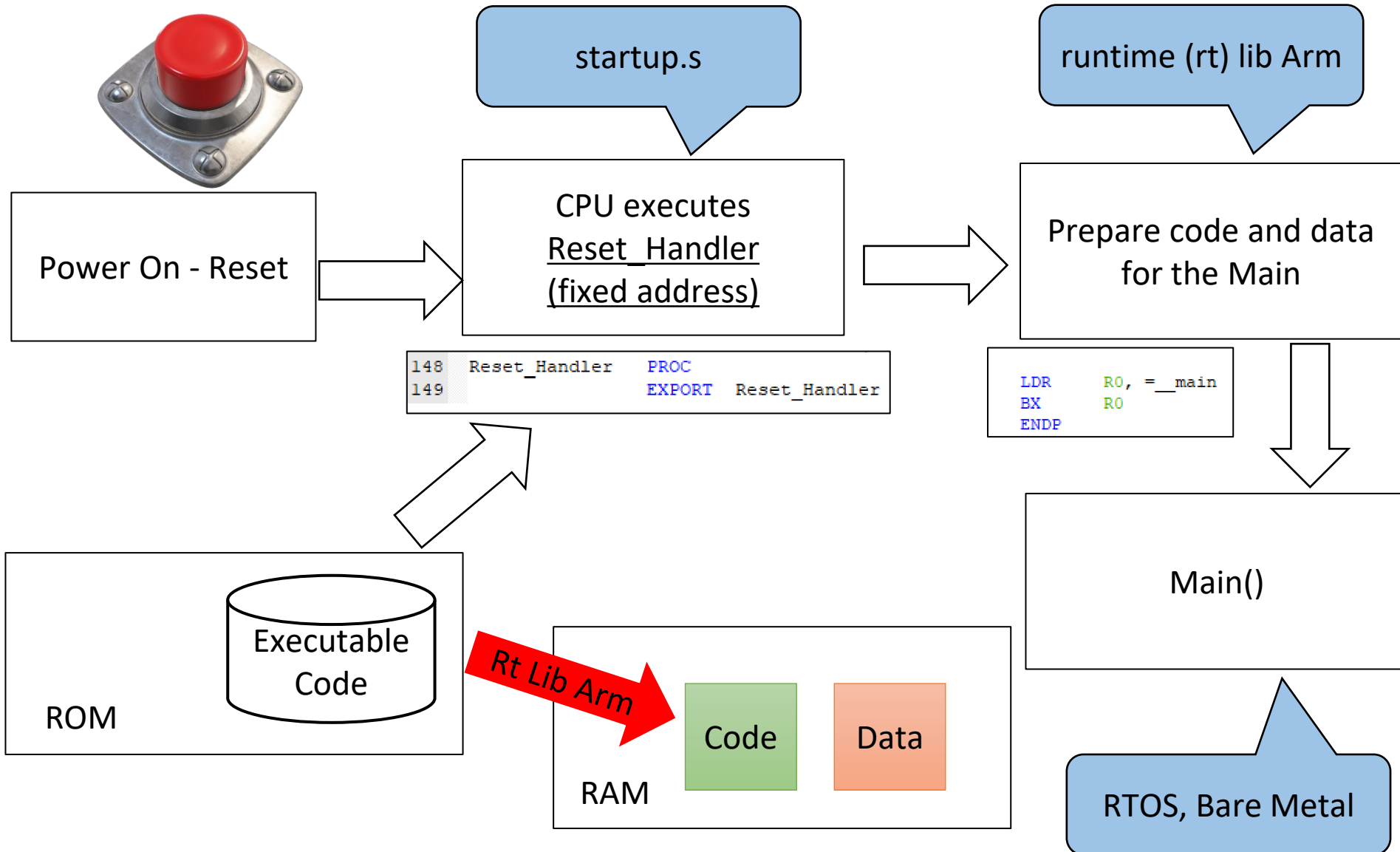
Rt Lib Arm

RAM

Code

Data

RTOS, Bare Metal



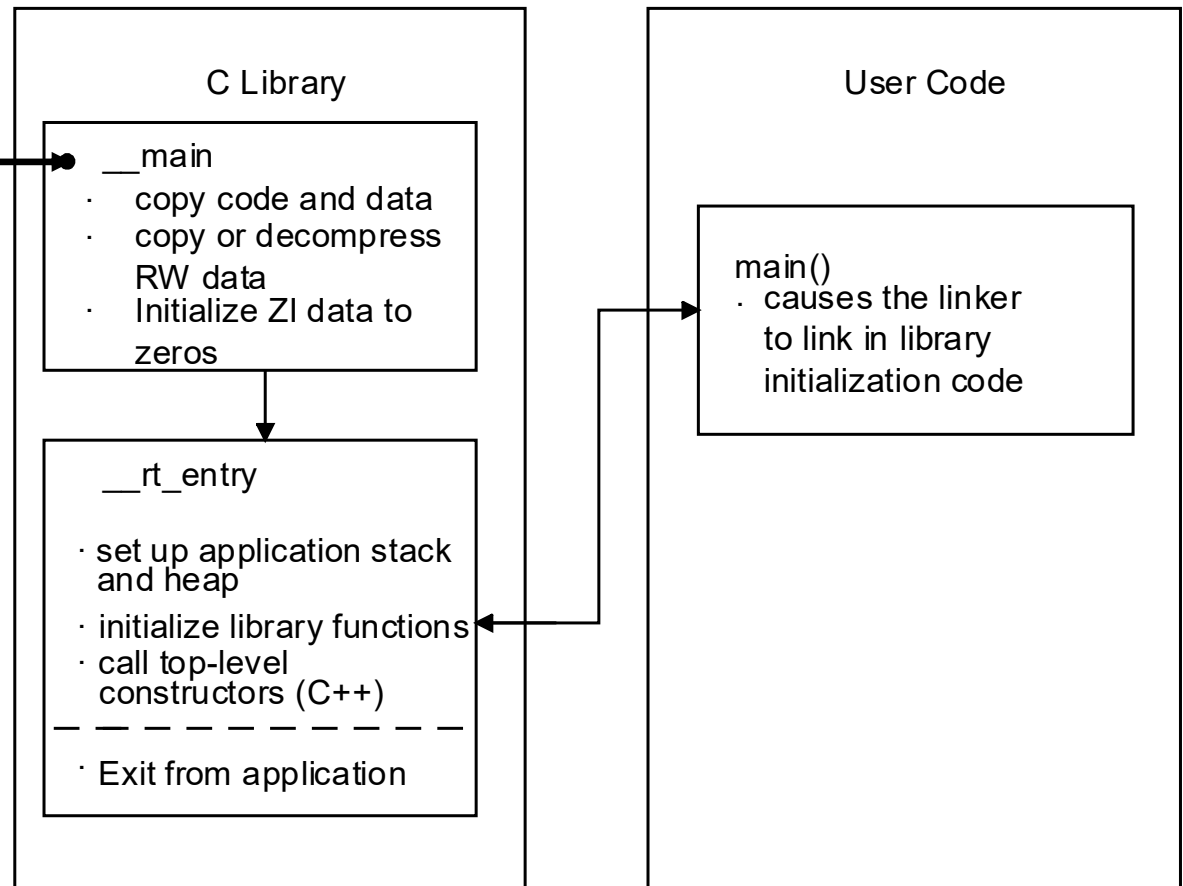
The Arm “magic secret sauce”

```
Reset_Handler  PROC
                 EXPORT Reset_Handler           [WEAK]
                 IMPORT  __main
                 LDR     R0, =__main
                 BX      R0
                 ENDP
```

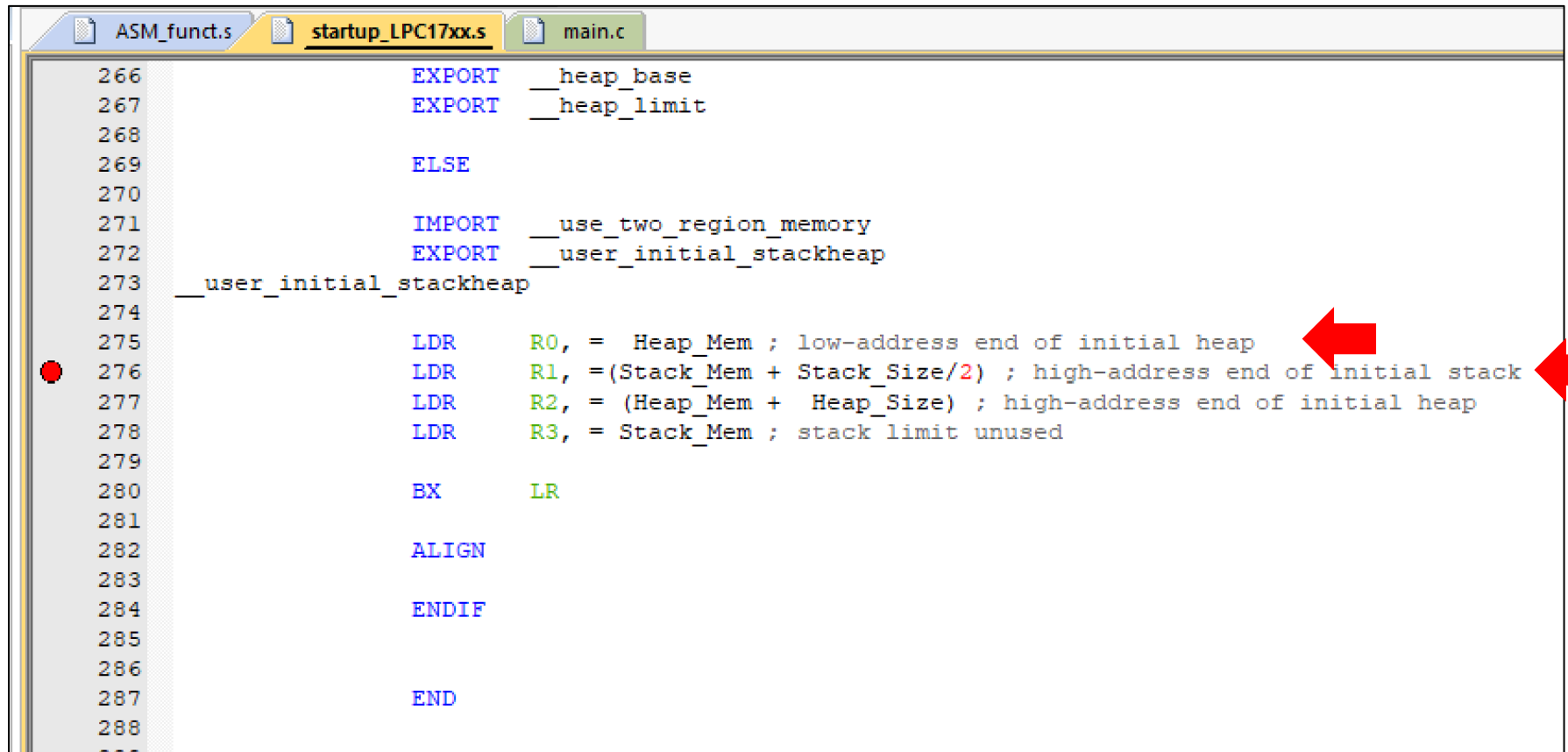
__main sets up the memory code and data

__rt_entry:

- calls *__user_initial_stackheap* to set up stacks and heap
- initializes lib functions and static data.
- calls top level constructors.



Setting up stack(s)



```
ASM_func.s startup_LPC17xx.s main.c
266      EXPORT __heap_base
267      EXPORT __heap_limit
268
269      ELSE
270
271      IMPORT __use_two_region_memory
272      EXPORT __user_initial_stackheap
273 __user_initial_stackheap
274
275      LDR     R0, = Heap_Mem ; low-address end of initial heap
276      LDR     R1, =(Stack_Mem + Stack_Size/2) ; high-address end of initial stack
277      LDR     R2, =(Heap_Mem + Heap_Size) ; high-address end of initial heap
278      LDR     R3, = Stack_Mem ; stack limit unused
279
280      BX     LR
281
282      ALIGN
283
284      ENDIF
285
286
287      END
288
289
```

References

- [Clang and the LLVM project](#)
- [Arm Compiler](#)
- [Arm Embedded Software Development](#)
- [Arm Image Structure and Generation](#)
- [Debugging With Arbitrary Record Formats \(DWARF\)](#) and [Executable and Linkable Format \(ELF\)](#)
- [Linux Boot Process](#)